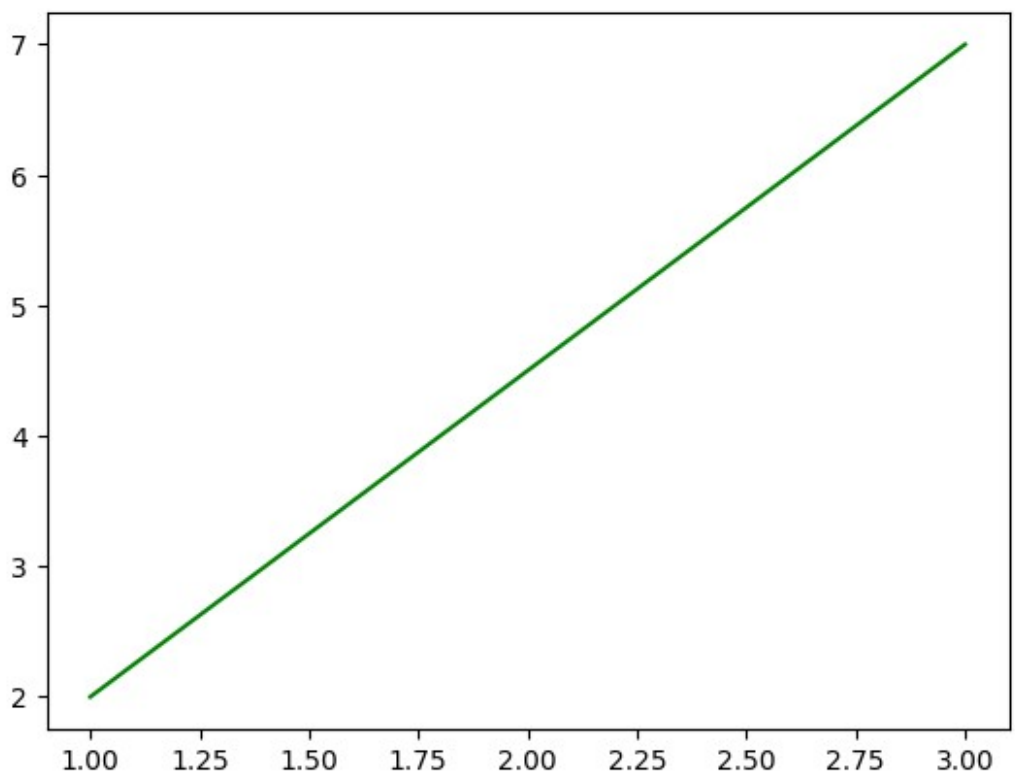
Regression using numpy (with pandas and pyplot)

File access required: In Colab this notebook requires first uploading files **Cities.csv** and **Players.csv** using the *Files* feature in the left toolbar. If running the notebook on a local computer, simply ensure these files are in the same workspace as the notebook.

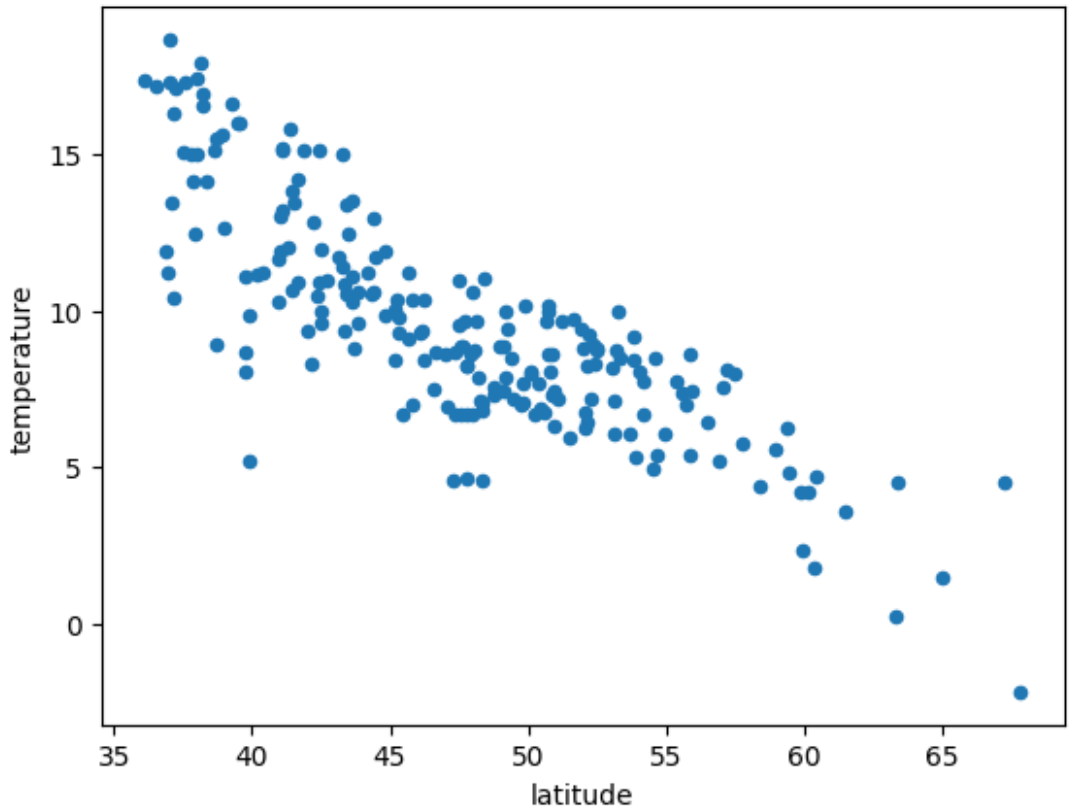
```
# Set-up
import csv
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

# Read Cities.csv into dataframe
f = open('Cities.csv')
cities = pd.read_csv(f)

cities.columns
```

```
Index(['city', 'country', 'latitude', 'longitude', 'temperature'],
dtype='object')
```

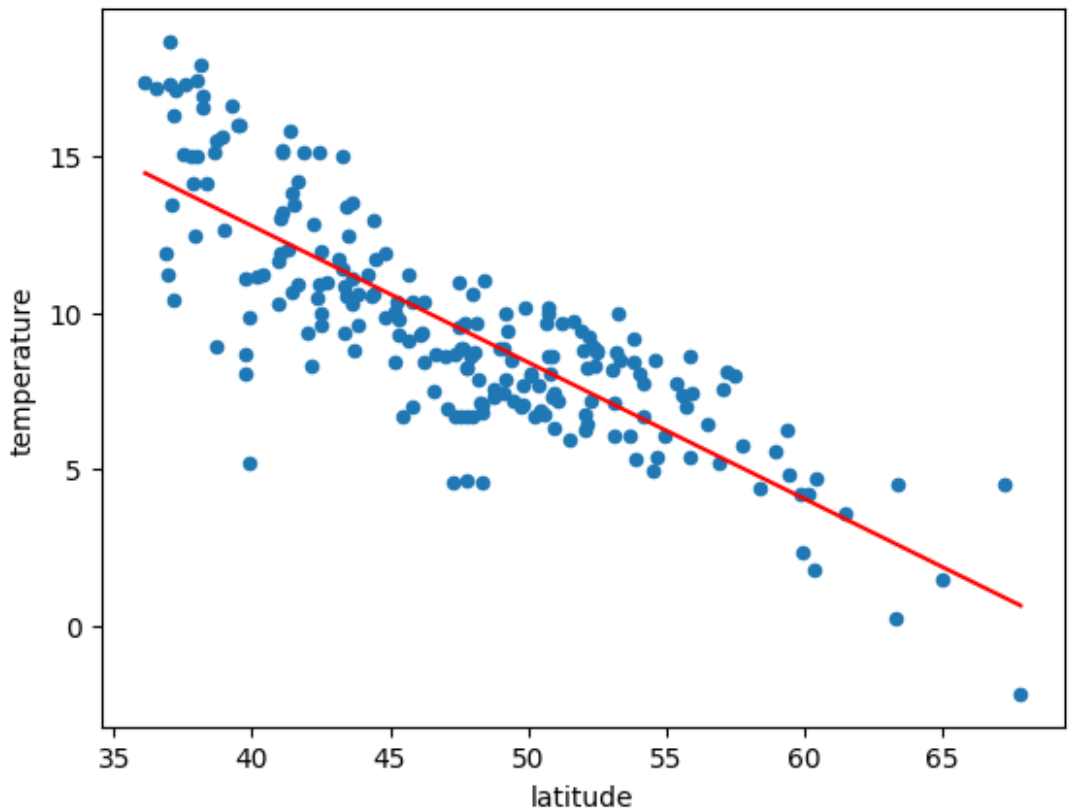
```
# Introduction to plotting a line
plt.plot([1,3], [2,7], color='green')
plt.show()
```



```
# Recall temperature versus latitude scatterplot
cities.plot.scatter('latitude', 'temperature')
plt.show()
```




```
# Add linear regression
cities.plot.scatter('latitude', 'temperature')

a,b = np.polyfit(cities.latitude, cities.temperature, 1) # Regression
line is y = mx + b

x1 = min(cities.latitude)
x2 = max(cities.latitude)

plt.plot([x1,x2], [a*x1 + b, a*x2 + b], color='red')
plt.show()
```



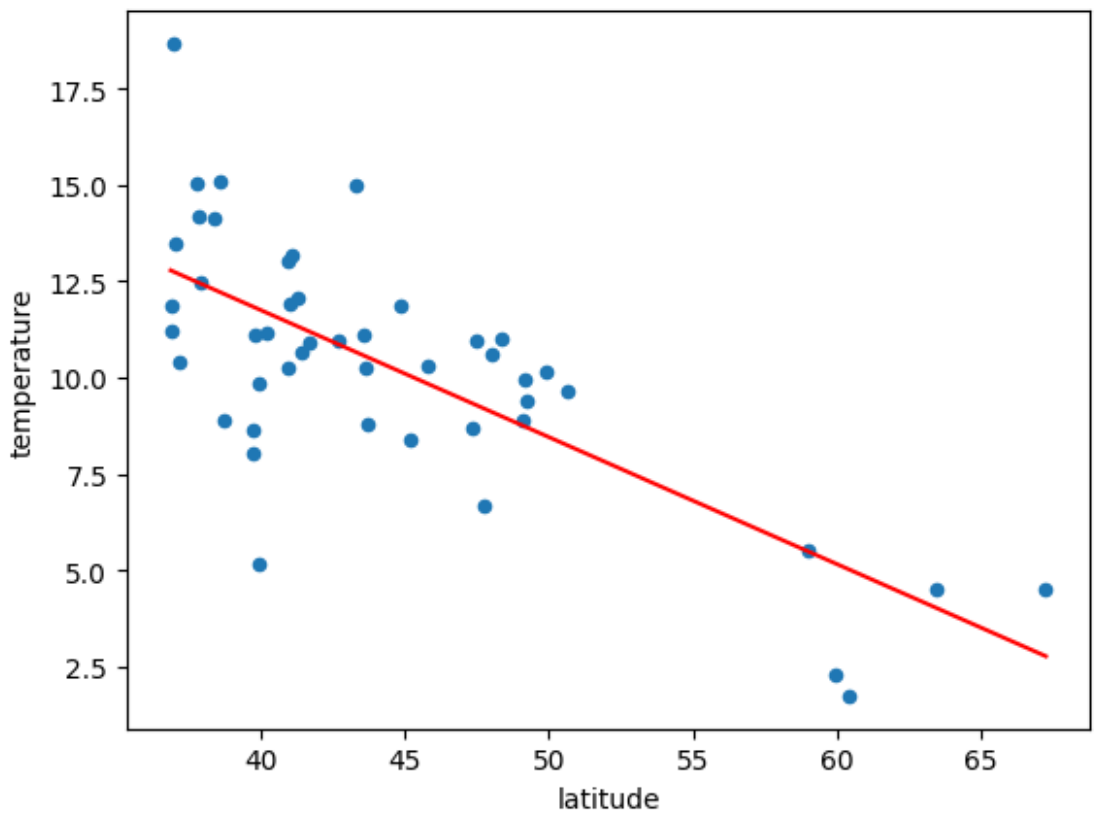

```
# Correlation coefficients (r values)
r = np.corrcoef(cities.latitude, cities.temperature)[1,0]
print('Correlation coefficient for temperature versus latitude:', r)

r = np.corrcoef(cities.longitude, cities.temperature)[1,0]
print('Correlation coefficient for temperature versus longitude:', r)
```

```
Correlation coefficient for temperature versus latitude: -
0.8241046320267099
Correlation coefficient for temperature versus longitude: -
0.16930750403617875
```

```
# Use linear regression for interactive temperature predictor
# Training data: compute and show latitude-temperature regression from
cities
# in Norway, France, and Turkey
train = cities[ (cities.country=='Norway') |
(cities.country=='France') | (cities.country=='Turkey')  ]
a,b = np.polyfit(train.latitude, train.temperature, 1) # y = mx + b =
y = ax + b --->>>>> slope: a; intercept: b
print('Training data:')
train.plot.scatter('latitude', 'temperature')
x1 = min(train.latitude)
x2 = max(train.latitude)
```

```
plt.plot([x1,x2], [a*x1 + b, a*x2 + b], color='red')
plt.show()
```

Training data:




```
train.head()
```

{"summary":"{\n \"name\": \"train\",\n \"rows\": 47,\n \"fields\": [\n {\n \"column\": \"city\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 47,\n \"samples\": [\n \"Manisa\",\n \"Sivas\",\n \"Malatya\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"country\",\n \"properties\": {\n \"dtype\": \"category\",\n \"num_unique_values\": 3,\n \"samples\": [\n \"Turkey\",\n \"France\",\n \"Norway\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"latitude\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 7.363122565395979,\n \"min\": 36.89,\n \"max\": 67.25,\n \"num_unique_values\": 46,\n \"samples\": [\n 58.97,\n 45.83,\n 38.37\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"longitude\",\n

\"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 15.879536278900984,\n   \"min\": -4.5,\n   \"max\": 41.93,\n   \"num_unique_values\": 47,\n   \"samples\": [\n   27.44,\n   37.03,\n   38.3\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"temperature\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 3.2983117790840213,\n   \"min\": 1.75,\n   \"max\": 18.67,\n   \"num_unique_values\": 46,\n   \"samples\": [\n   5.53,\n   10.32,\n   14.13\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"train"}

```python
# Now loop asking for a city name; find the city's latitude, use the regression
# to predict the temperature, then check against the actual temperature.

while True:
    name = input('Enter city name (or "quit" to quit): ')

    if name == 'quit': break

    city = cities[cities.city == name]
    if len(city) == 0:
        print('City not in dataset')
    else:
        # Use float() to convert dataframe element to value
        print('Predicted temperature:', a * float(city.latitude) + b) # y = mx + b = y = ax + b --->>>>> slope: a; intercept: b -> trained from 3 cities only
        print('Actual temperature:', float(city.temperature))
```

```
Enter city name (or "quit" to quit): Adana
```

```
/tmp/ipython-input-1798674423.py:14: FutureWarning: Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.iloc[0]) instead
  print('Predicted temperature:', a * float(city.latitude) + b) # y = mx + b = y = ax + b --->>>>> slope: a; intercept: b -> trained from 3 cities only
/tmp/ipython-input-1798674423.py:15: FutureWarning: Calling float on a single element Series is deprecated and will raise a TypeError in the future. Use float(ser.iloc[0]) instead
  print('Actual temperature:', float(city.temperature))
```

```
Predicted temperature: 12.750248828351404
Actual temperature: 18.67
Enter city name (or "quit" to quit): Amiens
Predicted temperature: 8.495921130358
```

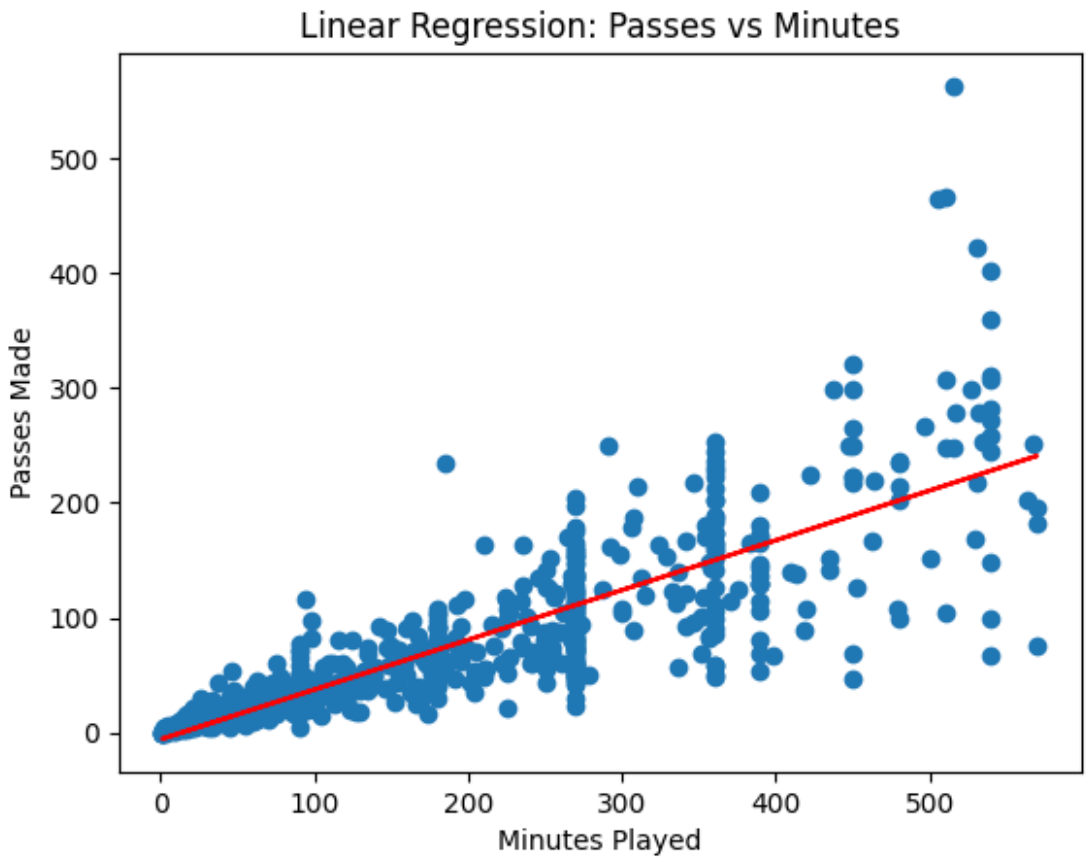
```
Actual temperature: 10.17
Enter city name (or "quit" to quit): quit
```

Your Turn: World Cup Data

```python
# Read Players.csv into dataframe
f = open('Players.csv')
players = pd.read_csv(f)

# Check column names
print(players.columns)
```

```
Index(['surname', 'team', 'position', 'minutes', 'shots', 'passes',
'tackles',
       'saves'],
      dtype='object')
```

```python
# From the players data, compute and plot a linear regression for
# passes made (y-axis) versus minutes played (x-axis).

from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Prepare data
X = players['minutes'].values.reshape(-1, 1)  # x-axis
y = players['passes'].values                  # y-axis

# Fit linear regression
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

# Plot
plt.scatter(X, y)
plt.plot(X, y_pred, color='red')
plt.xlabel('Minutes Played')
plt.ylabel('Passes Made')
plt.title('Linear Regression: Passes vs Minutes')
plt.show()
```




```python
# Show the correlation coefficients for four regressions:
# passes versus minutes, tackles versus minutes, shots versus minutes,
# and saves versus minutes. Make sure to label each one.

# List of stats to compare with minutes
stats = ['passes', 'tackles', 'shots', 'saves']

# Compute and print correlation coefficients
for stat in stats:
    corr = players['minutes'].corr(players[stat])
    print(f"Correlation between minutes and {stat}: {corr:.4f}")
```

```
Correlation between minutes and passes: 0.8151
Correlation between minutes and tackles: 0.6107
Correlation between minutes and shots: 0.4079
Correlation between minutes and saves: 0.2285
```

```python
# BONUS!
# Repeat the previous problem, except consider each position
separately.
# That is, show correlation coefficients separately for the forwards,
# midfielders, defenders, and goalkeepers. If you get a warning, you
# may hard-code the fact that only goalkeepers make saves.
```

```python
# Repeat correlation analysis for each position separately

# Define the positions we're analyzing
positions = ['forward', 'midfielder', 'defender', 'goalkeeper']

# For each position, compute correlations
for position in positions:
    # Filter players for this position
    position_df = players[players['position'] == position]

    print(f"\n=== {position.upper()} (n={len(position_df)}) ===")

    # Calculate and print correlations
    for stat in ['passes', 'tackles', 'shots']:
        correlation = position_df['minutes'].corr(position_df[stat])
        print(f"Minutes vs {stat}: {correlation:.4f}")

    # Special case for saves - only goalkeepers
    if position == 'goalkeeper':
        correlation =
position_df['minutes'].corr(position_df['saves'])
        print(f"Minutes vs saves: {correlation:.4f}")
```

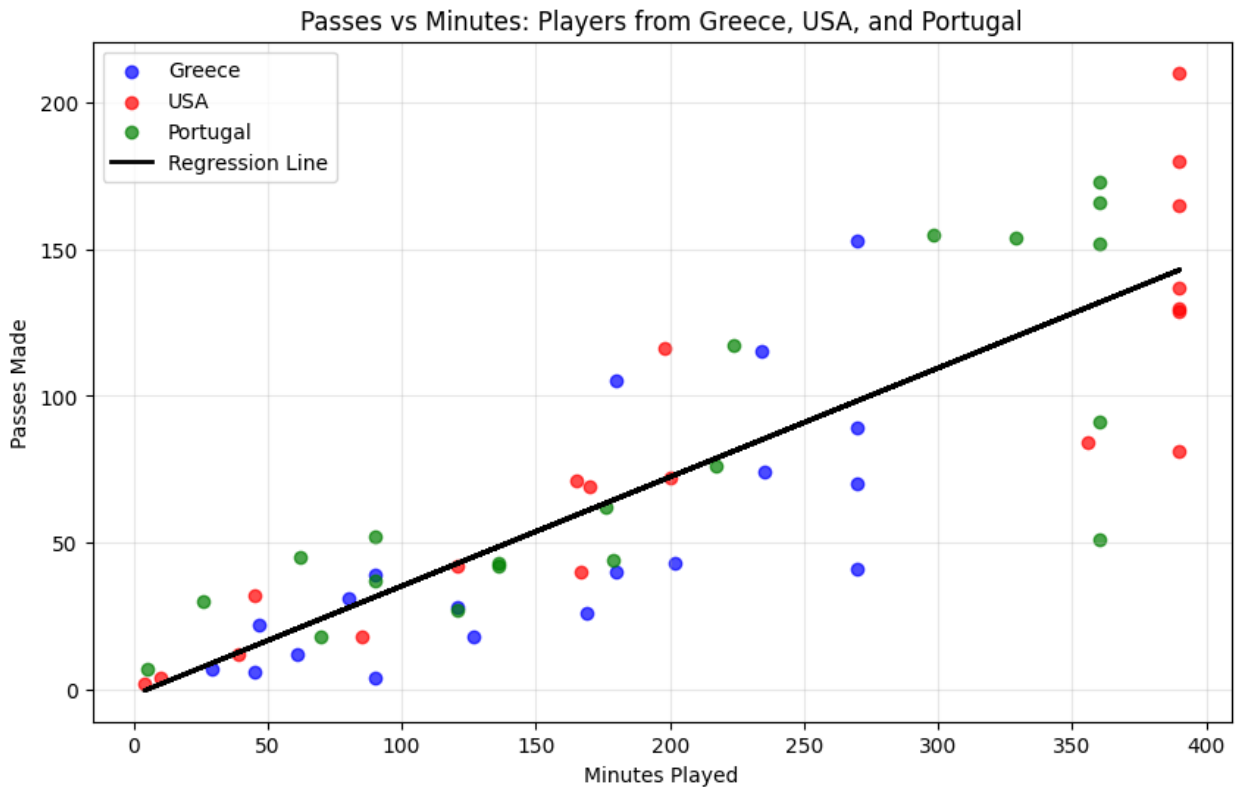
```
=== FORWARD (n=143) ===
Minutes vs passes: 0.8879
Minutes vs tackles: 0.5747
Minutes vs shots: 0.8482

=== MIDFIELDER (n=228) ===
Minutes vs passes: 0.8654
Minutes vs tackles: 0.7793
Minutes vs shots: 0.5360

=== DEFENDER (n=188) ===
Minutes vs passes: 0.8919
Minutes vs tackles: 0.6960
Minutes vs shots: 0.3624

=== GOALKEEPER (n=36) ===
Minutes vs passes: 0.8017
Minutes vs tackles: 0.2479
Minutes vs shots: -0.1893
Minutes vs saves: 0.6201
```

```python
# Use linear regression for interactive number-of-passes predictor
# Training data: compute and show minutes-passes regression for
players
# from Greece, USA, and Portugal
```

```python
# Filter players from Greece, USA, and Portugal
countries = ['Greece', 'USA', 'Portugal']
filtered_players = players[players['team'].isin(countries)]

# Check if we have data for these countries
if len(filtered_players) == 0:
    print("No players found from Greece, USA, or Portugal")
else:
    print(f"Found {len(filtered_players)} players from Greece, USA,
and Portugal")

    # Prepare data
    X = filtered_players['minutes'].values.reshape(-1, 1)
    y = filtered_players['passes'].values

    # Fit linear regression model
    model = LinearRegression()
    model.fit(X, y)

    # Get coefficients
    slope = model.coef_[0]
    intercept = model.intercept_

    print(f"\nRegression model: passes = {slope:.4f} * minutes +
{intercept:.4f}")
    print(f"R-squared: {model.score(X, y):.4f}")

    # Create predictions for visualization
    y_pred = model.predict(X)

    # Create scatter plot with regression line
    plt.figure(figsize=(10, 6))

    # Plot points for each country with different colors
    colors = {'Greece': 'blue', 'USA': 'red', 'Portugal': 'green'}

    for country in countries:
        country_data = filtered_players[filtered_players['team'] ==
country]
        if len(country_data) > 0:
            plt.scatter(country_data['minutes'],
country_data['passes'],
                        color=colors.get(country, 'gray'),
                        label=country, alpha=0.7)

    # Plot regression line
    plt.plot(X, y_pred, color='black', linewidth=2, label='Regression
Line')

    plt.xlabel('Minutes Played')
```

```
    plt.ylabel('Passes Made')
    plt.title('Passes vs Minutes: Players from Greece, USA, and
Portugal')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

    # Interactive predictor
    print("\n=== INTERACTIVE PREDICTOR ===")
    print("Enter minutes played to predict number of passes (or 'quit'
to exit):")

    while True:
        user_input = input("Minutes played: ")

        if user_input.lower() == 'quit':
            print("Exiting predictor...")
            break

        try:
            minutes = float(user_input)
            predicted_passes = model.predict([[minutes]])[0]
            print(f"Predicted passes for {minutes} minutes:
{predicted_passes:.1f}")
        except ValueError:
            print("Please enter a valid number or 'quit'")
        except:
            print("Error in prediction. Please try again.")
```

```
Found 58 players from Greece, USA, and Portugal

Regression model: passes = 0.3709 * minutes + -1.7533
R-squared: 0.7156
```




```
=== INTERACTIVE PREDICTOR ===
Enter minutes played to predict number of passes (or 'quit' to exit):
Minutes played: 12
Predicted passes for 12.0 minutes: 2.7
Minutes played: 4
Predicted passes for 4.0 minutes: -0.3
Minutes played: 16
Predicted passes for 16.0 minutes: 4.2
Minutes played: quit
Exiting predictor...
```

```python
# See surnames from Greece, USA, Portugal
print("\nSURNAMES FROM GREECE, USA, PORTUGAL:")
for country in ['Greece', 'USA', 'Portugal']:
    surnames = players[players['team'] == country]['surname'].unique()
    print(f"{country}: {len(surnames)} players")
    print(surnames[:20])
    print()
```

```
SURNAMES FROM GREECE, USA, PORTUGAL:
Greece: 19 players
['Avraam Papadopoulos' 'Charisteas' 'Gekas' 'Kapetanos' 'Karagounis'
 'Katsouranis' 'Kyrgiakos' 'Moras' 'Ninis' 'Papastathopoulos'
 'Patsatzoglou' 'Salpingidis' 'Samaras' 'Seitaridis' 'Spiropoulos'
```

```
 'Torosidis' 'Tziolis' 'Tzorvas' 'Vintra']

USA: 19 players
['Beasley' 'Bocanegra' 'Bornstein' 'Bradley' 'Buddle' 'Cherundolo'
'Clark'
 'Demerit' 'Dempsey' 'Donovan' 'Edu' 'Feilhaber' 'Findley' 'Gomez'
 'Holden' 'Howard' 'Jozy Altidore' 'Onyewu' 'Torres']

Portugal: 20 players
['Almeida' 'Amorim' 'Bruno Alves' 'Carvalho' 'Danny' 'Deco' 'Duda'
 'Eduardo' 'Ferreira' 'Fabio Coentrao' 'Liedson' 'Meireles' 'Mendes'
 'Miguel' 'Pepe' 'Ricardo Costa' 'Ronaldo' 'Simao' 'Tiago' 'Veloso']


# Now loop asking for a player's surname; find the number of minutes
the
# player played, use the regression to predict the number of passes
made,
# then check against the actual number.
#
# Once your predictor seems to be working, try surname Barry, Gomez,
or Pepe.
# You probably get an error; figure out the underlying reason for the
error,
# then modify your code so it prints a suitable message for surnames
like these,
# instead of raising an error.

print("=== PLAYER PASSES PREDICTOR ===")
print("Enter player surnames to predict passes (or 'quit' to exit)")
print("Try: Barry, Gomez, Pepe, or any surname from the dataset\n")

while True:
    surname = input("\nEnter player surname: ").strip()

    if surname.lower() == 'quit':
        print("Exiting predictor...")
        break

    # Search for player (case-insensitive)
    player_match = players[players['surname'].str.lower() ==
surname.lower()]

    # Check if player exists
    if len(player_match) == 0:
        print(f"Player '{surname}' not found in the dataset.")
        continue

    # Check if player is from Greece, USA, or Portugal
    player_row = player_match.iloc[0]
```

```
    player_country = player_row['team']

    if player_country not in ['Greece', 'USA', 'Portugal']:
        print(f"Player '{surname}' is from {player_country}, not
Greece, USA, or Portugal.")
        print("Our regression model was trained only on players from
Greece, USA, and Portugal.")
        print(f"Actual stats - Minutes: {player_row['minutes']},
Passes: {player_row['passes']}")
        continue

    # Get player's minutes
    minutes_played = player_row['minutes']
    actual_passes = player_row['passes']

    # Predict passes using our regression model
    predicted_passes = model.predict([[minutes_played]])[0]

    # Calculate difference
    difference = actual_passes - predicted_passes
    percent_error = (difference / predicted_passes * 100) if
predicted_passes != 0 else 0

    # Display results
    print(f"\nPlayer: {player_row['surname']} ({player_row['team']},
{player_row['position']})")
    print(f"Minutes played: {minutes_played}")
    print(f"Predicted passes: {predicted_passes:.1f}")
    print(f"Actual passes: {actual_passes}")
    print(f"Difference: {difference:+.1f} ({percent_error:+.1f}%)")

    if abs(percent_error) < 10:
        print("Prediction is very accurate!")
    elif abs(percent_error) < 25:
        print("Prediction is reasonably close.")
    else:
        print("Prediction is not very accurate for this player.")
```

```
=== PLAYER PASSES PREDICTOR ===
Enter player surnames to predict passes (or 'quit' to exit)
Try: Barry, Gomez, Pepe, or any surname from the dataset


Enter player surname: Beasley

Player: Beasley (USA, midfielder)
Minutes played: 10
Predicted passes: 2.0
Actual passes: 4
Difference: +2.0 (+104.6%)
```

```
Prediction is not very accurate for this player.

Enter player surname: Barry
Player 'Barry' is from England, not Greece, USA, or Portugal.
Our regression model was trained only on players from Greece, USA, and
Portugal.
Actual stats - Minutes: 264, Passes: 171

Enter player surname: quit
Exiting predictor...
```

```python
# BONUS!
# Repeat previous but use separate predictors for the four different
positions
# (forward,midfielder,defender,goalkeeper). Does it do better?
#
# Note: To extract a string value from a dataframe element use
df.iloc[0].element,
# e.g., if "player" is a one-row dataframe, then
player.iloc[0].position returns
# the player's position as a string

# Create regression models for each position
position_models = {}
for position in ['forward', 'midfielder', 'defender', 'goalkeeper']:
    pos_players = players[players['position'] == position]
    X = pos_players['minutes'].values.reshape(-1, 1)
    y = pos_players['passes'].values
    model = LinearRegression()
    model.fit(X, y)
    position_models[position] = model

# Interactive predictor
print("Enter player surnames to test position-specific models")
print("Type 'quit' to exit\n")

while True:
    surname = input("Surname: ").strip()
    if surname.lower() == 'quit':
        break

    # Find player using .iloc[0] as instructed
    player_match = players[players['surname'].str.lower() ==
surname.lower()]

    if len(player_match) == 0:
        print(f"'{surname}' not found\n")
        continue

    player = player_match.iloc[0]
```

```
    position = player['position']
    minutes = player['minutes']
    actual_passes = player['passes']

    # Use position-specific model
    if position in position_models:
        predicted_passes =
position_models[position].predict([[minutes]])[0]
        print(f"{surname} ({position}): {actual_passes} actual,
{predicted_passes:.1f} predicted")
    else:
        print(f"Unknown position: {position}")

    print()
```

```
Enter player surnames to test position-specific models
Type 'quit' to exit

Surname: Beasley
Beasley (midfielder): 4 actual, -0.8 predicted

Surname: quit
```